

AMAZON BEDROCK / デスクトップアプリの CLAUDE CODE

複雑な有価証券報告書データを、 AI で数分でダッシュボードに

Claude Code ハンズオン研修

題材: EDINET の XBRL

ゴール: 財務ダッシュ
ボード

体験: スキル化 → 横展
開

本日の流れ

1 まず触ってみる

2 今回のゴール

3 課題と準備

4 XBRL（覚えなくてよい）

5 1 社目を作る

6 スキルで横展開

7 会計基準への対応

8 CLAUDE.md とメモリ

9 自分でやってみる（課題）

10 まとめ

INTRODUCTION

講師 自己紹介

お名前 / 所属・役割 / Claude ・生成 AI でやってきたこと



今回の目的

日常業務を Claude で効率化し、

複雑な処理は AI に任せる感覚を持ち帰る。



複雑さは AI に

調べる・分岐・整形をまとめて任せる



手を動かすのは手元 PC

ファイル操作もコマンドも自分の PC で



一度作れば再利用

スキル化して別の会社にも横展開

実行環境 — モデルは AWS の Bedrock で動く

Amazon Bedrock

(AWS のフルマネージド生成 AI 基盤)

東京リージョン / 認証は Entra ID 連携

 **考える** = クラウド (Bedrock)

モデルはすべて AWS 上で実行される

 **動かす** = 手元 PC

ファイル操作・コマンドは自分の PC で実行

制約: MCP を利用した外部サービス連携は不可 (pip やデータ取得は手元 PC の通信次第で別問題)。

全体の流れ

ハンズオン 全体の流れ

EDINETの有報データ → Claude Codeで財務ダッシュボード → スキル化して別企業へ横展開



まず触ってみる

プロンプト

金の重量 (g) と純度 (%) を入力すると買取概算額を計算できる、単一 HTML のツールを作ってください。

- ・ 外部ライブラリは使わず、1 ファイル (index.html) で完結
- ・ 入力フォーム、計算はその場で表示
- ・ グラム単価は仮に 9,000 円 /g をコード内の定数に

作成後、ローカルで開いて動作を確認する手順も教えてください。

困った時の助け舟

— まず Claude に聞いてみる

— 完璧なプロンプトを目指さない (対話で育てる)

— 確認が入ることがある ← AI の時代でも人間が判断する場面

この一言で、動くツールができる

自然言語 → 動くもの → その場で確認

— 数十秒で、単一 HTML のツールが完成

— ブラウザで開いてすぐ動作確認

— 「作って→直して→また作る」を対話で回す

金 買取概算ツール

WARM-UP DEMO

重量 (g)

100

純度 (%)

99.9

買取概算額

¥899,100

グラム単価は 9,000円/g (コード内の定数、後で変更可)。純度で按分した概算値です。

今回のゴール

EDINET にある複雑な有価証券報告書ファイル（XBRL）から、**綺麗な財務ダッシュボード**を作る。

一完成イメージ → （右のサンプル）

一人が扱うには複雑すぎる XBRL を、AI でまとめて可視化

商用イメージの参照： [バフェットコード](#)

Copilot との住み分け： 市民開発・Office 連携は Copilot ／ 自律エージェント・複雑データ処理・コード生成は Claude Code 。



完成ダッシュボード（A R Eホールディングス）

配布物の共有と、作業フォルダの設定

① 配布物を受け取る

- 各社の有価証券報告書 XBRL 一式
- フォルダ名: data/< 社名 >_<EDINET コード >/
- (例) data/ トヨタ自動車 _E02144/
- コピペ用プロンプト集

② 作業フォルダを Claude に教える

- 配布データを置いたフォルダを開く
- そこを起点に Claude Code が読み書きする
- 以降は「このフォルダの○○を…」で指示できる

そもそも XBRL とは？

有価証券報告書などの数字を、**コンピュータが読めるようにタグ付けした形式。**

人にとっては

- タグや定義が大量で読みづらい
- 「どの数字が何か」が一目で分からない

AI にとっては

- タグが付いているので機械処理しやすい
- 狙った数字を正確に取り出せる

＝人には辛い形式だからこそ、AI が力を発揮する。

XBRL の構成

XBRLの構成 — タクソノミ（辞書・型） × インスタンス（実データ）

「勘定科目マスタ（タクソノミ）に沿って今期の数値（インスタンス）を記録したもの」がXBRL

タクソノミ（辞書・型）	どう読むか
.xsd	スキーマ 要素（項目）の定義・型
_pre	表示リンクベース 並び順・階層
_cal	計算リンクベース 合計＝内訳の加算
_def	定義リンクベース 軸（ディメンション）
_lab	ラベルリンクベース 日本語/英語名

インスタンス（.xbrl）

何がいくつか＝実データ

ファクト	値そのもの（多数）
コンテキスト	いつ・連結/個別
ユニット	単位（円・株・％）
DEI	会社情報・会計基準（IFRS/日本基準）

参照
各ファクトは要素定義を指す

1ファクトの分解（AREホールディングス 実例）

```
<jpcrp_cor:RevenueIFRSSummaryOfBusinessResults  
contextRef="CurrentYearDuration" unitRef="JPY">569992000000</...>
```

要素（コンセプト）	コンテキスト	ユニット	値（ファクト本体）
何の項目か Revenue... SummaryOf BusinessResults ＝ IFRS売上収益 （経営指標等） 名前空間 jpcrp_cor で 辞書上の定義に紐づく	いつ・連結/個別 CurrentYear Duration ＝ 当期（期間） 2025/4-2026/3・連結 Instant=時点/ Prior1Year=前期/ _NonConsolidated=個別	単位 JPY ＝ 円 ほかに pure（比率・株数） JPYPerShares（円/株）	数値そのもの 569992000000 ＝ 5,699.92 億円 decimals="-6" は 百万円未満を丸める意 値は円単位で格納。 表示時に ÷ 1億

- タクソノミ（辞書・型） + インスタンス（実データ .xbrl） = XBRL
- 数値の実体はファクト。「どの期・連結 / 個別か」はコンテキストで決まる
- 取得は 手段 1 = Web 手動 DL（キー不要）

⚠ 当日ハマる筆頭：証券コードは 5 桁（4 桁 + 末尾 0）

1つのデータはこう書かれている

```
<jpcrp_cor:RevenueIFRSSummaryOfBusinessResults  
  contextRef="CurrentYearDuration" unitRef="JPY">569992000000</...>
```

要するに → 「**当期の売上は約 5,700 億円**」と書いてあるだけ。

— 「いつの数字か」（当期／前期）と「単位」（円）が、数字と別の場所にタグで付いている

— 円で書かれているので、ダッシュボードでは ÷1 億して「億円」で表示する

— 1 つ 1 つは単純。でも種類が多く別々の場所にあるので、人が全部たどるのは大変 → だから AI に任せる

05 ・ 演習

さあ、実際にやってみる

ここからは配布データを使って、自分の手で動かします

プロンプトで複雑な処理を Claude に実行させる

プロンプト

data/ トヨタ自動車_E02144 の有価証券報告書 XBRL を解析して、主要財務をダッシュボードにして

この一言で、裏では次の処理が自動で走る：



コスト作法： 巨大な XBRL 本文を会話に貼らない。構成→対象タグ→スクリプト化→結果だけ確認。

別の会社も、すぐ作れる

「次はソニーで」「次は日立で」 — 何社もやりたくなりますよね。

一度やった作業は…

- 1 社目は試行錯誤
- でも手順はもう分かっている
- 毎回同じ指示を打ち直すのは大変

→ 手順を「道具」にまとめて再利用

スキル

という仕組み。2 社目以降は数分。

スキル＝手順書＋道具のパッケージ

- 作業の「やり方（手順書）」と「道具（スクリプト）」を1セットに
- 必要なときに Claude が自動で読み込んで実行
- 一度作れば、別の会社でもそのまま再利用・横展開
- 属人化を防ぐ“内製の資産”になる

```
skills/edinet-xbrl-dashboard/  
├ SKILL.md           ... 使い方の説明書（本体）  
├ scripts/           ... 道具（Python）  
│   ├── extract_financials.py  
│   └── build_dashboard.py  
└ assets/dashboard_template.html
```

スキルは、プロンプトで作れる

プロンプト

今やった「有価証券報告書 XBRL → 主要財務の抽出 → ダッシュボード生成」の一連の手順を、再利用できるスキルにまとめてください。

- ・入力（XBRL フォルダ）と出力（`dashboard.html`）を明記
- ・会社固有の値はハードコードせず、別の会社でもそのまま使えるように作成後、このスキルの使い方（呼び出し方）も教えてください。

ポイント

一試行錯誤の成果を、そのまま“再利用できる資産”に変える
一次からは「このスキルで、この会社を」と一言でよい

ちなみに… 会計基準は 1 種類じゃない

別の会社でやってみると——

同じ「売上」でも、会社によって科目名が違う。

IFRS 採用の会社

一売上収益 / 税引前利益 / 親会社所有者持分
…

日本基準 採用の会社

一売上高 / 経常利益 / 純資産 …

配布17 社にも、IFRS と 日本基準が混在している。

会計基準が違っても、 Claude がそのまま対応

人がやると…

- 基準を判定し、拾う項目を切り替える
- 銀行など特殊な様式にも対応…
- 知らずに実行して、エラーで気づく

Claude なら…

- どの会社を指しても同じ操作で OK
- 違いに“気づいて”対応してくれる
- IFRS の会社も銀行も、続けて流せば両方出る

立て付け

- 人間は「知らずに実行 → エラー」になりがちなところを、Claude は先に気づいて確認してくれる
- 締めに「1社だけ原文と検算」=任せつつ、人が確かめる

気づいた対応は、スキルに反映する

プロンプト

日本基準の会社で必要になった科目の対応を、edinet-xbrl-dashboard スキルに反映して、次からは自動で切り替わるようにしてください。

スキルが“育つ”

- 対応を一度スキルに書けば恒久化
- 一次の会社からは、もう気にしなくてよい

これが内製の効き目

- 使うほどスキルが賢くなる
- チームの“資産”として貯まっていく

よりよいアウトプットのための仕組み

毎回うまく動かすために、Claude に“覚えさせる”仕組みが2つある。

CLAUDE.md

人が書く「決まり事」

—データの場所・採用ルール・出力先などを明記

—毎回指示しなくても品質が安定

メモリ

Claude が対話から自動で覚える

—与えた修正や好みを学習ノートに蓄積

—使うほど自分の好みに馴染む

CLAUDE.md

作業フォルダに置くと起動時に自動で読み込まれる「前提メモ」。

```
# このプロジェクトについて（例）
- 対象データ： EDINET の有報 XBRL（ ./data/... ）
- 採用方針： 連結・通期を優先。表示は億円
- 出力： ./output/ に成果物（単一 HTML）
- 会計基準は自動判定。銀行は解釈に注意
```

読み込み順（後ほど優先）

- 管理ポリシー（組織）
- ~/.claude/CLAUDE.md（個人）
- 上位ディレクトリ
- ./CLAUDE.md（プロジェクト）
- ./CLAUDE.local.md

見せ方：「置く前／置いた後」で出力の質を比較。

メモリ（対話から自動で覚える）

```
~/.claude/projects/<repo>/memory/  
├ MEMORY.md          (インデックス)  
└ feedback_*.md 等   (トピック別)
```

—/memory で確認・編集。既定で ON、マシンローカル
—例：「金額は億円」と一度直すと次から反映される

CLAUDE.md との使い分け

	CLAUDE.md	メモリ
書き手	人	Claude
中身	前提・ルール	学習・好み
性質	git で共有・再現	ローカル・自動

再現性は **CLAUDE.md + スキル**、快適さは **メモリ**。

課題 ― 手を動かして広げる



好きな企業を追加

配布データ以外の会社でも
同じスキルで作ってみる



機能を足してみる

バフェットコードを参考に
指標やグラフを追加



企業どうしを比較

2社を並べて
強み・弱みを見比べる

まとめ

- 人間が実行するには複雑な処理も、Claude なら自動化できる
- アイデア次第で、普段の事務処理も大きく時間短縮できる
- ビジュアライズして、これまで気づけなかった価値を生み出せる

まず触る → 1社作る → スキルで横展開 → 自分の業務へ